

第10章 初识 STM32 固件库

在上一章中，我们构建了几个控制 GPIO 外设的函数，算是实现了函数库的雏形，但 GPIO 还有很多功能函数我们没有实现，而且 STM32 芯片不仅仅只有 GPIO 这一个外设。如果我们想要亲自完成这个函数库，工作量是非常巨大的。ST 公司提供的标准软件库，包含了 STM32 芯片所有寄存器的控制操作，我们直接学习如何使用 ST 标准库，会极大地方便控制 STM32 芯片。接下来我们简单的分析下 ST 官方的固件库的组成部分。

10.1 CMSIS 标准及库层次关系

因为基于 Cortex 系列芯片采用的内核都是相同的，区别主要为核外的片上外设的差异，这些差异却导致软件在同内核，不同外设的芯片上移植困难。为了解决不同的芯片厂商生产的 Cortex 微控制器软件的兼容性问题，ARM 与芯片厂商建立了 CMSIS 标准(Cortex MicroController Software Interface Standard)。

所谓 CMSIS 标准，实际是新建了一个软件抽象层，具体见见图 10-1。

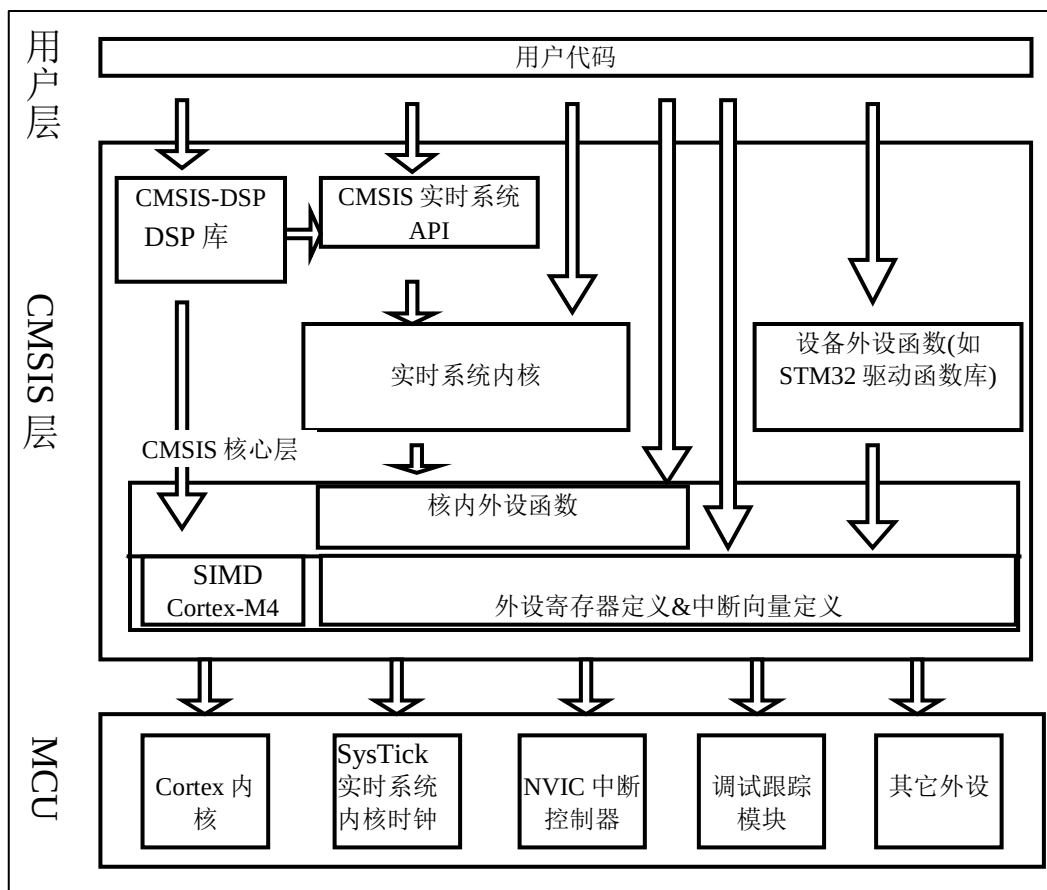


图 10-1 CMSIS 架构

CMSIS 标准中最主要的为 CMSIS 核心层，它包括了：

- ❑ 内核函数层：其中包含用于访问内核寄存器的名称、地址定义，主要由 ARM 公司提供。
- ❑ 设备外设访问层：提供了片上的核外外设的地址和中断定义，主要由芯片生产商提供。

可见 CMSIS 层位于硬件层与操作系统或用户层之间，提供了与芯片生产商无关的硬件抽象层，可以为接口外设、实时操作系统提供简单的处理器软件接口，屏蔽了硬件差异，这对软件的移植是有极大的好处的。STM32 的库，就是按照 CMSIS 标准建立的。

10.1.1 库目录、文件简介

STM32 标准库可以从官网获得，也可以直接从本书的配套资料得到。本书讲解的例程全部采用 1.8.0 库文件。以下内容请大家打开 STM32 标准库文件配合阅读。

解压库文件后进入其目录：

“STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\”

软件库各文件夹的内容说明见图 10-2。



图 10-2 ST 标准库 目录：STM32F4xx_DSP_StdPeriph_Lib_V1.5.1\

- ❑ Libraries：文件夹下是驱动库的源代码及启动文件。
- ❑ Project：文件夹下是用驱动库写的例子和工程模板。
- ❑ Utilities：包含了基于 ST 官方实验板的例程，以及第三方软件库，如 emwin 图形软件库、fatfs 文件系统。
- ❑ MCD-ST Liberty...：库文件的 License 说明。
- ❑ Release_Note.html：库的版本更新说明。
- ❑ stm32f4xx_dsp_stdperiph...：库帮助文档，这是一个已经编译好的 HTML 文件，主要讲述如何使用驱动库来编写自己的应用程序。说得形象一点，这个 HTML 就是告诉我们：ST 公司已经为你写好了每个外设的驱动了，想知道如何运用这些例子就来向我求救吧。不幸的是，这个帮助文档是英文的，这对很多英文不好的朋友来说是一个很大的障碍。但这里要告诉大家，英文仅仅是一种工具，绝对不能让它成为我们学习的障碍。其实这些英文还是很简单的，我们需要的是拿下它的勇气。

在使用库开发时，我们需要把 libraries 目录下的库函数文件添加到工程中，并查阅库帮助文档来了解 ST 提供的库函数，这个文档说明了每一个库函数的使用方法。

进入 Libraries 文件夹看到，关于内核与外设的库文件分别存放在 CMSIS 和 STM32F4xx_StdPeriph_Driver 文件夹中。

先看看 CMSIS 文件夹。

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Libraries\CMSIS\文件夹下内容见图 10-3。



图 10-3 CMSIS 文件夹内容 目录：Libraries\CMSIS\

其中 Device 与 Include 中的文件是我们使用得最多的，先讲解这两个文件夹中的内容。

1. Include 文件夹

在 Include 文件夹中包含了的是位于 CMSIS 标准的核内设备函数层的 Cortex-M 核通用的头文件，它们的作用是为那些采用 Cortex-M 核设计 SOC 的芯片商设计的芯片外设提供一个进入内核的接口，定义了一些内核相关的寄存器(类似我们前面写的 stm32f4xx.h 文件，但定义的是内核部分的寄存器)。这些文件在其它公司的 Cortex-M 系列芯片也是相同的。至于这些功能是怎样用源码实现的，可以不用管它，只需把这些文件加进我们的工程文件即可，有兴趣的朋友可以深究，关于内核的寄存器说明，需要查阅《cortex_m4_Technical Reference Manual》及《Cortex™-M4 内核参考手册》文档，《STM32 参考手册》只包含片上外设说明，不包含内核寄存器。

我们写 STM32F4 的工程，必须用到其中的四个文件：core_cm4.h、core_cmFunc.h、corecmInstr.h、core_cmSimd.h，其它的文件是属于其它内核的，还有几个文件是 DSP 函数库使用的头文件。

core_cm4.c 文件有一些与编译器相关条件编译语句，用于屏蔽不同编译器的差异。里面包含了一些跟编译器相关的信息，如：“__CC_ARM” (本书采用的 RVMDK、KEIL)，“__GNUC__” (GNU 编译器)、“ICC Compiler” (IAR 编译器)。这些不同的编译器对于 C 嵌入汇编或内联函数关键字的语法不一样，这段代码统一使用“__ASM、__INLINE”宏来定义，而在不同的编译器下，宏自动更改到相应的值，实现了差异屏蔽，见代码清单 10-1。

代码清单 10-1: core_cm4.c 文件中对编译器差异的屏蔽

```
1 #if defined ( __CC_ARM )
2 #define __ASM __asm /*!< asm keyword for ARM Compiler */
3 #define __INLINE inline /*!< inline keyword for ARM Compiler*/
4 #define __STATIC_INLINE static inline
5
6 #elif defined ( __GNUC__ )
7 #define __ASM __asm /*!< asm keyword for GNU Compiler */
8 #define __INLINE inline /*!< inline keyword for GNU Compiler */
9 #define __STATIC_INLINE static inline
10
11 #elif defined ( __ICCARM__ )
12 #define __ASM __asm /*!< asm keyword for IAR Compiler */
13 /*!< inline keyword for IAR Compiler. */
14 #define __STATIC_INLINE static inline
15 #define __INLINE inline
16
17 #elif defined ( __TMS470__ )
18 #define __ASM __asm /*!< asm keyword for TI CCS Compiler */
19 #define __STATIC_INLINE static inline
20
21 #elif defined ( __TASKING__ )
22 #define __ASM __asm /*!< asm keyword for TASKING Compiler */
23 #define __INLINE inline /*!< inline keyword for TASKING Compiler */
24 #define __STATIC_INLINE static inline
25
26 #elif defined ( __CSMC__ )
27 #define __packed
28 #define __ASM __asm /*!< asm keyword for COSMIC Compiler */
29 /*use -pc99 on compile line !< inline keyword for COSMIC Compiler */
30 #define __INLINE inline
31 #define __STATIC_INLINE static inline
32
33 #endif
```

使用 RVMDK 编译器时的
嵌入汇编与内联函数
的关键字形式

使用 GNU 编译器时的
形式

较重要的是在 core_cm4.c 文件中包含了“stdint.h”这个头文件，这是一个 ANSI C 文件，是独立于处理器之外的，就像我们熟知的 C 语言头文件“stdio.h”文件一样。位于 RVMDK 这个软件的安装目录下，主要作用是提供一些类型定义。见代码清单 10-2。

代码清单 10-2: stdint.c 文件中的类型定义

```
1. /* exact-width signed integer types */
2. typedef signed char int8_t;
3. typedef signed short int int16_t;
4. typedef signed int int32_t;
5. typedef signed __int64 int64_t;
6.
7. /* exact-width unsigned integer types */
8. typedef unsigned char uint8_t;
9. typedef unsigned short int uint16_t;
10. typedef unsigned int uint32_t;
11. typedef unsigned __int64 uint64_t;
```

这些新类型定义屏蔽了在不同芯片平台时，出现的诸如 int 的大小是 16 位，还是 32 位的差异。所以在我们以后的程序中，都将使用新类型如 uint8_t、uint16_t 等。

在稍旧版的程序中还经常会出现如 `u8`、`u16`、`u32` 这样的类型，分别表示的无符号的 8 位、16 位、32 位整型。初学者碰到这样的旧类型感觉一头雾水，它们定义的位置在 `STM32f4xx.h` 文件中。建议在以后的新程序中尽量使用 `uint8_t`、`uint16_t` 类型的定义。

`core_cm4.c` 跟启动文件一样都是底层文件，都是由 ARM 公司提供的，遵守 CMSIS 标准，即所有 CM4 芯片的库都带有这个文件，这样软件在不同的 CM4 芯片的移植工作就得以简化。

2. Device 文件夹

在 Device 文件夹下的是具体芯片直接相关的文件，包含启动文件、芯片外设寄存器定义、系统时钟初始化功能的一些文件，这是由 ST 公司提供的。

❑ `system_stm32f4xx.c` 文件

文件目录：\Libraries\CMSIS\Device\ST\STM32F4xx\Source\Templates

这个文件包含了 STM32 芯片上电后初始化系统时钟、扩展外部存储器用的函数，例如我们前两章提到供启动文件调用的“`SystemInit`”函数，用于上电后初始化时钟，该函数的定义就存储在 `system_stm32f4xx.c` 文件。STM32F429 系列的芯片，调用库的这个 `SystemInit` 函数后，系统时钟被初始化为 180MHz，如有需要可以修改这个文件的内容，设置成自己所需的时钟频率。

❑ 启动文件

文件目录：Libraries\CMSIS\Device\ST\STM32F4xx\Source\Templates

在这个目录下，还有很多文件夹，如“ARM”、“gcc_ride7”、“iar”等，这些文件夹下包含了对应编译平台的汇编启动文件，在实际使用时要根据编译平台来选择。我们使用的 MDK 启动文件在“ARM”文件夹中。其中的“`startup_stm32f40xx.s`”即为 STM32F407 芯片的启动文件，前面两章工程中使用的启动文件就是从这里复制过去的。如果使用其它型号的芯片，要在此处选择对应的启动文件，如 STM32F446 型号使用“`startup_stm32f446xx.s`”文件。

❑ `stm32f4xx.h` 文件

文件目录：Libraries\CMSIS\Device\ST\STM32F4xx\Include

`stm32f4xx.h` 这个文件非常重要，是一个 STM32 芯片底层相关的文件。它是我们前两章自己定义的“`stm32f4xx.h`”文件的完整版，包含了 STM32 中所有的外设寄存器地址和结构体类型定义，在使用到 STM32 标准库的地方都要包含这个头文件。

CMSIS 文件夹中的主要内容就是这样，接下来我们看看 `STM32F4xx_StdPeriph_Driver` 文件夹。

3. STM32F4xx_StdPeriph_Driver 文件夹

文件目录：Libraries\STM32F4xx_StdPeriph_Driver

进入 `libraries` 目录下的 `STM32F4xx_StdPeriph_Driver` 文件夹，见图 10-4。

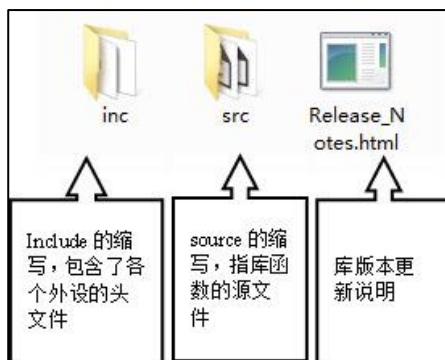


图 10-4 外设驱动

STM32F4xx_StdPeriph_Driver 文件夹下有 inc (include 的缩写) 跟 src (source 的缩写) 这两个文件夹, 这里的文件属于 CMSIS 之外的、芯片片上外设部分。src 里面是每个设备外设的驱动源程序, inc 则是相对应的外设头文件。src 及 inc 文件夹是 ST 标准库的主要内容, 甚至不少人直接认为 ST 标准库就是指这些文件, 可见其重要性。

在 src 和 inc 文件夹里的就是 ST 公司针对每个 STM32 外设而编写的库函数文件, 每个外设对应一个 .c 和 .h 后缀的文件。我们把这类外设文件统称为: stm32f4xx_ppp.c 或 stm32f4xx_ppp.h 文件, PPP 表示外设名称。如在上一章中我们自建的 stm32f4xx_gpio.c 及 stm32f4xx_gpio.h 文件, 就属于这一类。

如针对模数转换(ADC)外设, 在 src 文件夹下有一个 stm32f4xx_adc.c 源文件, 在 inc 文件夹下有一个 stm32f4xx_adc.h 头文件, 若我们开发的工程中用到了 STM32 内部的 ADC, 则至少要把这两个文件包含到工程里。见图 10-5。



图 10-5 驱动的源文件及头文件

这两个文件夹中，还有一个很特别的 misc.c 文件，这个文件提供了外设对内核中的 NVIC(中断向量控制器)的访问函数，在配置中断时，我们必须把这个文件添加到工程中。

4. stm32f4xx_it.c、stm32f4xx_conf.h 文件

文件目录：

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Project\STM32F4xx_StdPeriph_Templates

在这个文件目录下，存放了官方的一个库工程模板，我们在用库建立一个完整的工程时，还需要添加这个目录下的 stm32f4xx_it.c、stm32f4xx_it.h、stm32f4xx_conf.h 这三个文件。

stm32f4xx_it.c: 这个文件是专门用来编写中断服务函数的，在我们修改前，这个文件已经定义了一些系统异常(特殊中断)的接口，其它普通中断服务函数由我们自己添加。但是我们怎么知道这些中断服务函数的接口如何写？是不是可以自定义呢？答案当然不是的，这些都有可以在汇编启动文件中找到，在学习中断和启动文件的时候我们会详细介绍

stm32f4xx_conf.h: 这个文件被包含进 stm32f4xx.h 文件。ST 标准库支持所有 STM32F4 型号的芯片，但有的型号芯片外设功能比较多，所以使用这个配置文件根据芯片型号增减 ST 库的外设文件，具体见代码清单 10-3，针对 STM32F429 和 STM32F427 型号芯片的差异，它们实际包含不一样的头文件，我们通过宏来指定芯片的型号。

代码清单 10-3 stm32f4xx_conf.h 文件配置软件库

```
1
2 #if defined (STM32F429_439xx) || defined(STM32F446xx)
3 #include "stm32f4xx_cryp.h"
4 #include "stm32f4xx_hash.h"
5 #include "stm32f4xx_rng.h"
6 #include "stm32f4xx_can.h"
7 #include "stm32f4xx_dac.h"
8 #include "stm32f4xx_dcmi.h"
9 #include "stm32f4xx_dma2d.h"
10 #include "stm32f4xx_fmc.h"
11 #include "stm32f4xx_ltdc.h"
12 #include "stm32f4xx_sai.h"
13 #endif /* STM32F429_439xx || STM32F446xx */
14
15 #if defined (STM32F427_437xx)
16 #include "stm32f4xx_cryp.h"
17 #include "stm32f4xx_hash.h"
18 #include "stm32f4xx_rng.h"
19 #include "stm32f4xx_can.h"
20 #include "stm32f4xx_dac.h"
21 #include "stm32f4xx_dcmi.h"
22 #include "stm32f4xx_dma2d.h"
23 #include "stm32f4xx_fmc.h"
24 #include "stm32f4xx_sai.h"
25 #endif /* STM32F427_437xx */
26
27 #if defined (STM32F40_41xxx)
```

stm32f4xx_conf.h 这个文件还可配置是否使用“断言”编译选项，见代码清单 10-4。

代码清单 10-4 断言配置

```
1 #ifndef USE_FULL_ASSERT
2
3 /**
```

```
4  * @brief The assert_param macro is used for parameters check.
5  * @param expr: If expr is false, it calls assert_failed function
6  *   which reports the name of the source file and the source
7  *   line number of the call that failed.
8  *   If expr is true, it returns no value.
9  * @retval None
10 */
11 #define assert_param(expr) ((expr) ? (void)0 : assert_failed((uint8_t
                                                                    *)__FILE__, __LINE__))
12 /* Exported functions ----- */
13 void assert_failed(uint8_t* file, uint32_t line);
14 #else
15 #define assert_param(expr) ((void)0)
16 #endif /* USE_FULL_ASSERT */
```

在 ST 标准库的函数中，一般会包含输入参数检查，即上述代码中的“assert_param”宏，当参数不符合要求时，会调用“assert_failed”函数，这个函数默认是空的。

实际开发中使用断言时，先通过定义 USE_FULL_ASSERT 宏来使能断言，然后定义“assert_failed”函数，通常我们会让它调用 printf 函数输出错误说明。使能断言后，程序运行时会检查函数的输入参数，当软件经过测试，可发布时，会取消 USE_FULL_ASSERT 宏来去掉断言功能，使程序全速运行。

10.1.2 库各文件间的关系

前面向大家简单介绍了各个库文件的作用，库文件是直接包含进工程即可，丝毫不用修改，而有的文件就要我们在使用的时候根据具体的需要进行配置。接下来从整体上把握一下各个文件在库工程中的层次或关系，这些文件对应到 CMSIS 标准架构上。见图 10-6。

STM32 库文件结构

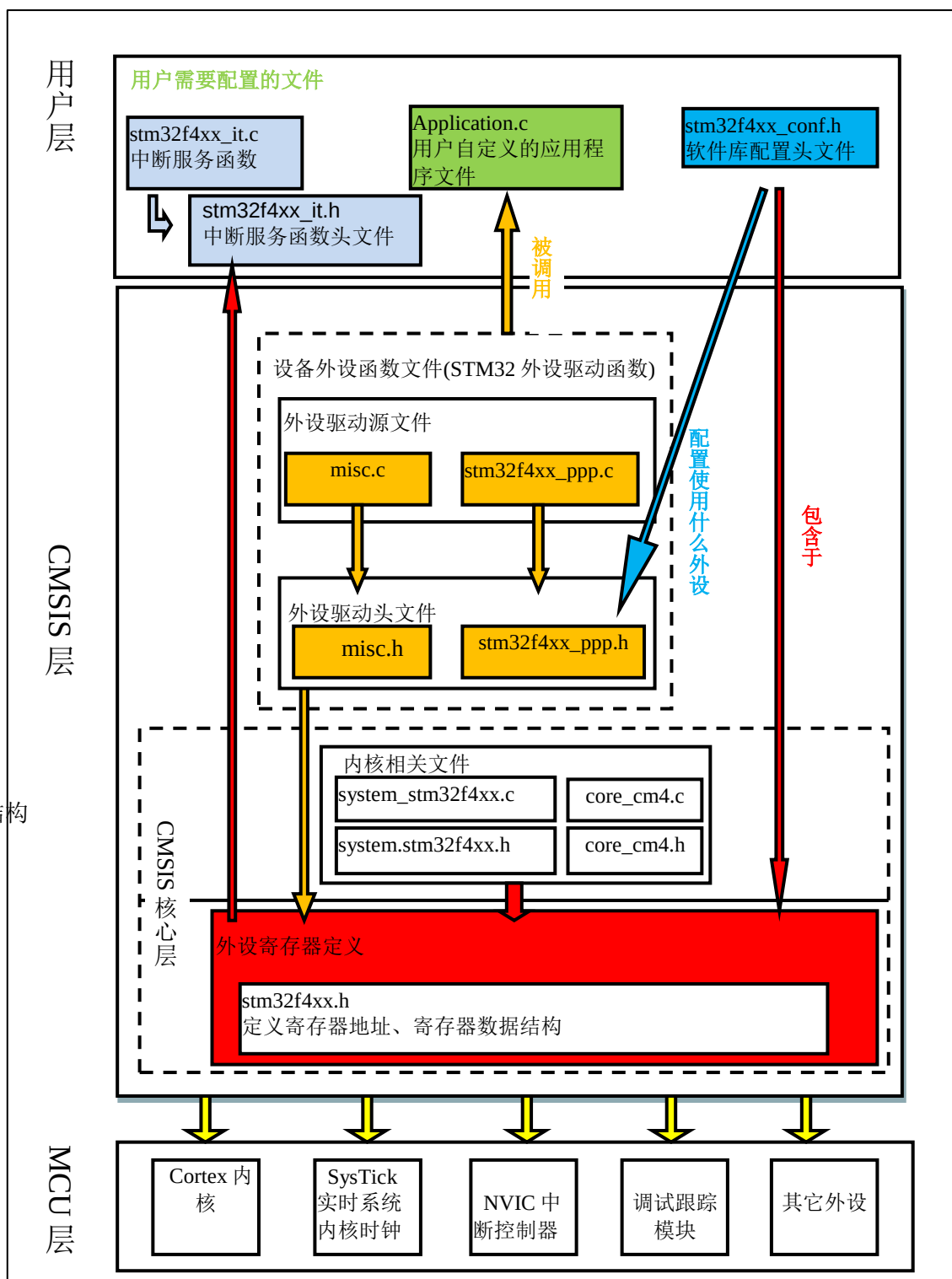


图 10-6 库各文件关系

图 10-6 描述了 STM32 库各文件之间的调用关系，这个图省略了 DSP 核和实时系统层部分的文件关系。在实际的使用库开发工程的过程中，我们把位于 CMSIS 层的文件包含进工程，除了特殊系统时钟需要修改 `system_stm32f4xx.c`，其它文件丝毫不用修改，也不建议修改。

对于位于用户层的几个文件，就是我们在使用库的时候，针对不同的应用对库文件进行增删（用条件编译的方法增删）和改动的文件。

10.2 使帮助文档

我坚信，授之以鱼不如授之以渔。官方资料是所有关于 STM32 知识的源头，所以在本小节介绍如何使用官方资料。官方的帮助手册，是最好的教程，几乎包含了所有在开发过程中遇到的问题。这些资料已整理到了本书附录资料中。

10.2.1 常用官方资料

❑ 《STM32F4xx 中文参考手册》

这个文件全方位介绍了 STM32 芯片的各种片上外设，它把 STM32 的时钟、存储器架构、及各种外设、寄存器都描述得清清楚楚。当我们对 STM32 的外设感到困惑时，可查阅这个文档。以直接配置寄存器方式开发的话，查阅这个文档寄存器部分的频率会相当高，但这样效率太低了。

❑ 《STM32F4xx 英文数据手册》

本文档相当于 STM32 的 datasheet，包含了 STM32 芯片所有的引脚功能说明及存储器架构、芯片外设架构说明。后面我们使用 STM32 其它外设时，常常需要查找这个手册，了解外设对应到 STM32 的哪个 GPIO 引脚。

❑ 《Cortex™-M4 内核参考手册》

本文档由 ST 公司提供，主要讲解 STM32 内核寄存器相关的说明，例如系统定时器、中断等寄存器。这部分的内容是《STM32F4xx 参考手册》没涉及到的内核部分的补充。相对来说，本文档虽然介绍了内核寄存器，但不如以下两个文档详细，要了解内核时，可作为以下两个手册的配合资料使用。

❑ 《Cortex-M3 权威指南》、《cortex_m4_Technical Reference Manual》。

这两个手册是由 ARM 公司提供的，它详细讲解了 Cortex 内核的架构和特性，要深入了解 Cortex-M 内核，这是首选，经典中的经典，其中 Cortex-M3 版本有中文版，方便学习。因为 Cortex-M4 内核与 Cortex-M3 内核大部分相同，可用它来学习，而 Cortex-M4 新增的特性，则必须参考《cortex_m4_Technical Reference Manual》文档了，目前只有英文版。

❑ 《stm32f4xx_dsp_stdperiph_lib_um.chm》

这个就是本章提到的库的帮助文档，在使用库函数时，我们最好通过查阅此文件来了解标准库提供了哪些外设、函数原型或库函数的调用的方法。也可以直接阅读源码里面的函数的函数说明。

10.2.2 初识库函数

所谓库函数，就是 STM32 的库文件中为我们编写好的函数接口，我们只要调用这些库函数，就可以对 STM32 进行配置，达到控制目的。我们可以不知道库函数是如何实现的，但我们调用函数必须要知道函数的功能、可传入的参数及其意义、和函数的返回值。

于是，有读者就问那么多函数我怎么记呀？我的回答是：会查就行了，哪个人记得了那么多。所以我们学会查阅库帮助文档是很有必要的。

打开库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》见图 10-7

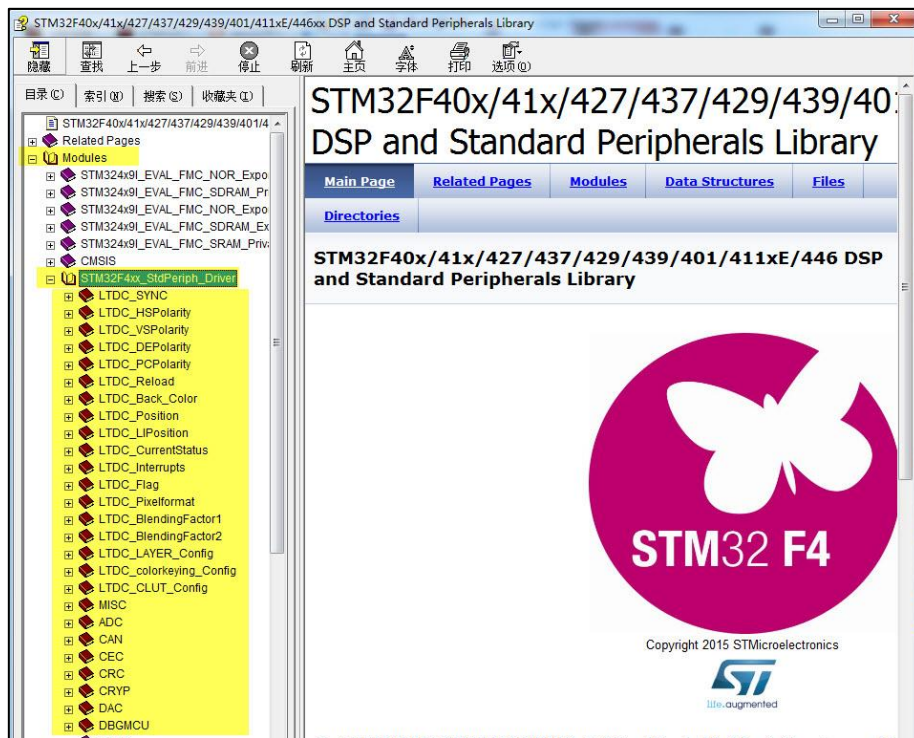


图 10-7 库帮助文档

层层打开文档的目录标签：

标签目录：Modules\STM32F4xx_StdPeriph_Driver\

可看到 STM32F4xx_StdPeriph_Driver 标签下有很多外设驱动文件的名字 MISC、ADC、BKP、CAN 等标签。

我们试着查看 GPIO 的“位设置函数 GPIO_SetBits”看看，打开标签：

标签目录：Modules\STM32F4xx_StdPeriph_Driver\GPIO\Functions\GPIO_SetBits 见图 10-8。

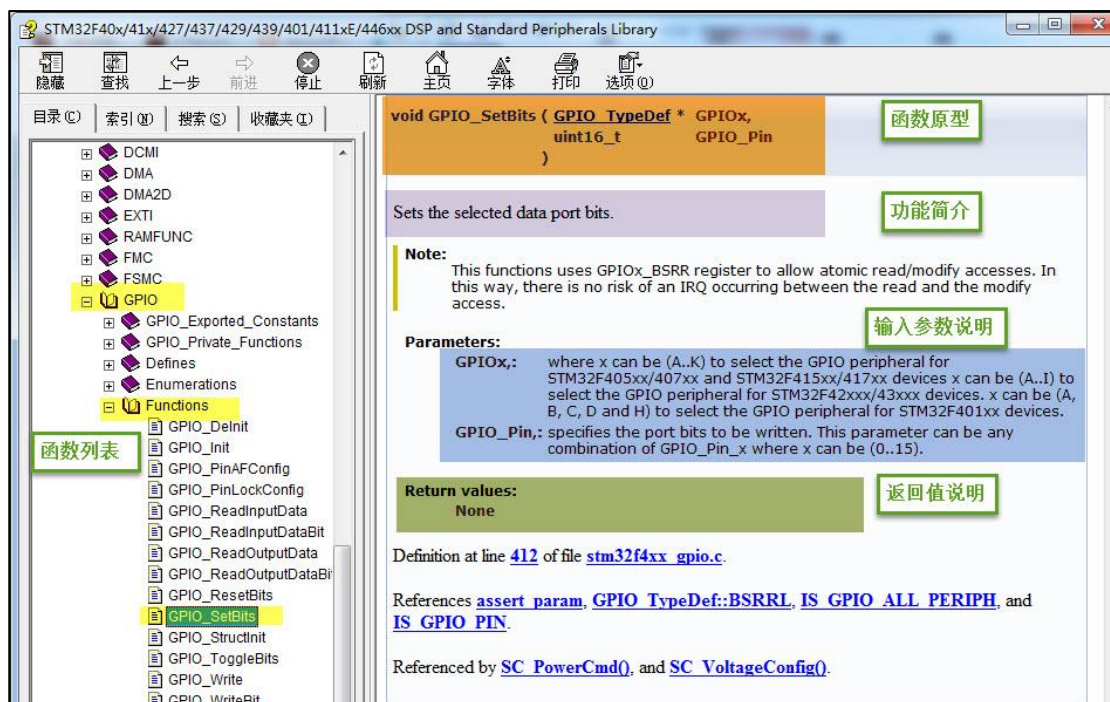


图 10-8 库帮助文档的函数说明

利用这个文档，我们即使没有去看它的具体源代码，也知道要怎么利用它了。

如 GPIO_SetBits，函数的原型为 void GPIO_SetBits(GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)。它的功能是：输入一个类型为 GPIO_TypeDef 的指针 GPIOx 参数，选定要控制的 GPIO 端口；输入 GPIO_Pin_x 宏，其中 x 指端口的引脚号，指定要控制的引脚。

其中输入的参数 GPIOx 为 ST 标准库中定义的自定义数据类型，这两个传入参数均为结构体指针。初学时，我们并不知道如 GPIO_TypeDef 这样的类型是什么意思，可以点击函数原型中带下划线的 GPIO_TypeDef 就可以查看这个类型的声明了。

就这样初步了解了一下库函数，读者就可以发现 STM32 的库是写得很优美的。每个函数和数据类型都符合见名知义的原则，当然，这样的名称写起来特别长，而且对于我们来说要输入这么长的英文，很容易出错，所以在开发软件的时候，在用到库函数的地方，直接把库帮助文档中的函数名称复制粘贴到工程文件就可以了。而且，配合 MDK 软件的代码自动补全功能，可以减少输入量。

有的用户觉得使用库文档麻烦，也可以直接查阅 STM32 标准库的源码，库帮助文档的说明都是根据源码生成的，所以直接看源码也可以了解函数功能。